CS240: Database Systems

Lecture 05:

SQL

Outline

- **Introduction**
- Data Manipulation Language (DML)
  - Simple SQL Queries
  - Nested SQL Queries
  - Aggregation and Grouping
  - Others, More Example: Relational Algebra and SQL
- Advanced Concepts (DDL)
  - Introduction to View
  - Introduction to Trigger

Introduction

- Structured Query Language (SQL) is the most widely used commercial relational database language.
- SQL continues to evolve in response to the changing need.
  - The current ANSI/ISO standard for SQL is called SQL:1999. The previous standard is SQL-92.

Introduction

- The SQL language has several aspects to it
  - The Data Manipulation Language (DML)
    - The subset of SQL that allows users to pose queries and to insert, delete, and modify rows.
  - The Data Definition Language (DDL)
    - The subset of SQL that supports the creation, deletion, and modification to definitions for tables and views.
  - Triggers and Advanced Integrity Constraints
    - The new SQL:1999 standard includes support for triggers, which are actions executed by the DBMS whenever changes to the database meet conditions specified in the trigger

Outline

- Introduction
- **Data Manipulation Language (DML)**
  - **Simple SQL Queries**
  - Nested SQL Queries
  - Aggregation and Grouping
  - Others, More Example: Relational Algebra and SQL
- Advanced Concepts (DDL)
  - Introduction to View
  - Introduction to Assertion/Trigger

Retrieval Queries in SQL

- SQL has one basic statement for retrieving information from a database; the SELECT statement
  - This is **NOT** the same as the SELECT (σ) operation of the relational algebra

Basic SQL Query

- The basic form of SQL

SELECT	[DISTINCT] *target-list*
FROM	*relation-list*
WHERE	*qualification*

- relation-list
  - A list of relation names (possibly with a range-variable after each name).
- target-list
  - A list of attributes of relations in relation-list.

SELECT	[DISTINCT] *target-list*
FROM	*relation-list*
WHERE	*qualification*

- qualification
  - Comparisons (*attr op const* or *attr1 op attr2*, where *op* is one of $\geq, \leq, <, =, >, \neq$) combined using AND, OR and NOT.
- DISTINCT
  - an optional keyword indicating that the answer should not contain duplicates.
- Default is that duplicates are not eliminated.

```
SELECT    [DISTINCT] target-list
FROM      relation-list
WHERE     qualification
```

- SELECT clause
  - specifies columns to be retained in the result.
- FROM clause
  - specifies a cross-product of tables.
- WHERE clause (optional)
  - specifies selection conditions on the tables mentioned in the FROM clause.
- An SQL query intuitively corresponds to a relational algebra expression involving selections, projections, and cross-products.

SELECT DISTINCT $a_1, a_2, \ldots, a_n$

FROM $R_1, R_2, \ldots, R_m$

WHERE P

||

$$\Pi_{a1, a2, \ldots, an} (\sigma_P (R_1 \times R_2 \times \ldots \times R_m))$$

Basic SQL Query: Processing Steps

SELECT	[DISTINCT] *target-list*
FROM	*relation-list*
WHERE	*qualification*

- Conceptual Evaluation Strategy
  - Compute the cross-product of *relation-list*.
  - Discard resulting tuples if they fail *qualifications*.
  - Delete attributes that are not in *target-list*.
  - If *DISTINCT* is specified, eliminate duplicate rows.

Example: SELECT-PROJECT

- Find the names of all branches in the loan relation.

```
SELECT branch-name
FROM Loan
```

Loan

branch_name	loan_number	amount
MUST	222	2
CTM	333	1
MUST	777	2

Result

branch_name
MUST
CTM
MUST

Example: DISTINCT Keyword

- Find the names of all branches in the loan relation and remove duplicate.

```
SELECT DISTINCT branch-name
FROM Loan
```

Loan

branch_name	loan_number	amount
MUST	222	2
CTM	333	1
MUST	777	2

Result

branch_name
MUST
CTM

$$\Pi_{branch\_name}\ (\ Loan\ )$$

The Rename Operation in SQL

- Renaming relations and attributes using the ***AS*** clause:

  old-name ***AS*** new-name

- For convenience, usually, “as” can be omitted in FROM clause.

```
SELECT L.branch-name AS name
FROM   Loan AS L
```

- ***AS*** in SQL is used to define a logical variable, called range variable, in the context of either FROM or explicit JOIN.

Find the names of sailors who have reserved boat number 103

Sailors

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Reserves

sid	bid	day
22	101	10/10/05
58	103	11/12/05

Rename & Cross Product

```
SELECT  S.sname
FROM    Sailors S, Reserves R
WHERE   S.sid=R.sid AND R.bid=103
```

JOIN Condition

S.sid	sname	rating	age	R.sid	bid	day
22	dustin	7	45.0	22	101	10/10/05
22	dustin	7	45.0	58	103	11/12/05
31	lubber	8	55.5	22	101	10/10/05
31	lubber	8	55.5	58	103	11/12/05
58	rusty	10	35.0	22	101	10/10/05
58	rusty	10	35.0	58	103	11/12/05

Row remains after selection.

Result

$\Pi_{sname}\ \sigma_{bid=103,\ S.sid=R.sid}\ (\text{Sailors} \times \text{Reserves})$

$\Pi_{sname}\ \sigma_{bid=103}\ (\text{Sailors} \bowtie \text{Reserves})$

The Rename Operation in SQL

- A Note on Range Variables
  - Really needed only if the same relation appears twice in the FROM clause.
- The previous query can also be written as:

```
SELECT  S.sname
FROM    Sailors S, Reserves R
WHERE  S.sid=R.sid AND R.bid=103
```

It is good style, however, to use range variables always!

or

```
SELECT  sname
FROM    Sailors, Reserves
WHERE  Sailors.sid=Reserves.sid AND bid=103
```

Expressions and Strings

```
SELECT  S.age, age1 = S.age−5,  2 * S.age AS age2
FROM    Sailors S
WHERE   S.sname LIKE 'B_%B'
```

- Illustrates use of arithmetic expressions and string pattern matching: Find triples (of ages of sailors and two fields defined by expressions) for sailors whose names begin and end with B and contain at least three characters.
  - **AS** and **=** are two ways to name fields in result.
  - **LIKE** is used for string matching.
  - '_' stands for any one character
  - `**%**' stands for 0 or more arbitrary characters.

Examples: Simple SQL

- Given the following schema:

Sailors (sid; integer, sname: string, rating:integer, age: real)

Boats (bid: integer, bname: string, color: string)

Reserves (sid: integer, bid: integer, day: date)

Sailors

sid	sname	rating	age

Boats

bid	bname	color

Reserves

sid	bid	day

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q1: Find the sids of sailors who have reserved a red boat.

```
SELECT  R.sid
FROM  Boat B, Reserves R
WHERE  B.bid = R.bid AND B.color = 'red'
```

- Q2: Find the names of sailors who have reserved a red boat.

```
SELECT  S.sname
FROM  Sailors S, Reserves R, Boat B
WHERE  S.sid = R.sid AND R.bid = B.bid AND
          B.color = 'red'
```

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q3: Find the colors of boats reserved by Lubber.

```
SELECT  B.color
FROM  Sailors S, Reserves R, Boat B
WHERE  S.sid = R.sid AND R.bid = B.bid AND
          S.name = 'Lubber'.
```

(In general, there may be more than one sailor called Lubber. In this case, it will return the colors of boats reserved by all such Lubber's).

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q4: Find the names of sailors who have reserved at least one boat.

```
SELECT  S.name
FROM    Sailors S, Reserves R
WHERE   S.sid = R.sid
```

Ordering the Display of Tuples

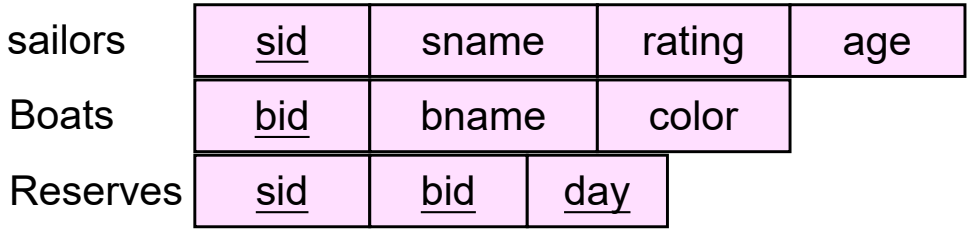
- The ORDER BY clause is used to sort the tuples in a query result based on the values of some attribute(s)
  - ORDER BY Ai [ASC/DESC]
    - ***ASC*** for ascending order (default)
    - ***DESC*** for descending order
    - SQL must perform a sort to fulfill an ORDER BY request. Since sorting a large number of tuples may be costly, it is desirable to sort only when necessary.
- For example: List in alphabetic order the names of all sailors

```
SELECT    S.name
FROM      Sailors S
ORDER BY  S.name
```

Set Operations

- The set operation UNION, INTERSECT, and EXCEPT operate on relations and correspond to the relational algebra operations $\cup$, $\cap$ and –.
- The set operations apply only to union compatible relations.
- Each of the above operations *automatically eliminates duplicates*; to retain all duplicates, we can use union all, intersect all and except all.

Examples



- Q5: Find sid's of sailors who've reserved a red *or* a green boat
- **UNION**: compute the union of any two union-compatible sets of tuples (which are themselves the result of SQL queries).
- If we replace **OR** by **AND** in the first version, what do we get?

```
SELECT  DISTINCT R.sid
FROM  Boats B, Reserves R
WHERE  R.bid=B.bid
        AND (B.color='red' OR B.color='green')
```

```
SELECT  R.sid
FROM  Boats B, Reserves R
WHERE  R.bid=B.bid
            AND B.color='red'
UNION
SELECT  R.sid
FROM  Boats B, Reserves R
WHERE  R.bid=B.bid
            AND B.color='green'
```

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q6: Find sid's of sailors who've reserved a red *and* a green boat
- **INTERSECT:** compute the intersection of any two union-compatible sets of tuples.

```
SELECT  R.sid
FROM  Boats B, Reserves R
WHERE   R.bid=B.bid
            AND B.color='red'
INTERSECT
SELECT  R.sid
FROM  Boats B, Reserves R
WHERE   R.bid=B.bid
            AND B.color='green'
```

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q7: Find sid's of all sailors who've reserved red boat but not green boat.

```
SELECT  R.sid
FROM  Boats B, Reserves R
WHERE  R.bid=B.bid AND B.color='red'

EXCEPT

SELECT  R.sid
FROM  Boats B, Reserves R
WHERE  R.bid=B.bid AND B.color='green'
```

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q8: Find sid's of all sailors who have a rating of 10 *or* reserved boat 104

```
SELECT  S.sid
FROM  Sailor S
WHERE  S.rating = 10

UNION

SELECT  R.sid
FROM  Reserves R
WHERE  R.bid=104
```

Outline

- Introduction
- **Data Manipulation Language (DML)**
  - Simple SQL Queries
  - **Nested SQL Queries**
  - Aggregation and Grouping
  - Others, More Example: Relational Algebra and SQL
- Advanced Concepts (DDL)
  - Introduction to View
  - Introduction to Assertion/Trigger

Nested Queries

- A nested query is a query that has another query embedded within it.
- The embedded query is called a subquery.
- The embedded query can be a nested query itself.
- Every SQL statement returns: a relation/set in the result OR Null OR A single atomic value
- We can replace a value or set of values with a SQL statement (ie., a subquery)
  - Illegal if the subquery returns the wrong type for the comparison
- Queries may have very deeply nested structures.

Uncorrelated Nested Queries

- Q9: Find names of sailors who've reserved boat #103:

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN  (SELECT  R.sid
                  FROM  Reserves R
                  WHERE  R.bid=103)
```

- A very powerful feature of SQL: a **WHERE** clause can itself contain an SQL query! (Actually, so can FORM clauses.)
- To find sailors who've not reserved #103, use **NOT IN**.
- To understand semantics of nested queries, think of a *nested loops* evaluation: *For each Sailors tuple, check the qualification by computing the subquery.*

Correlated Nested Queries

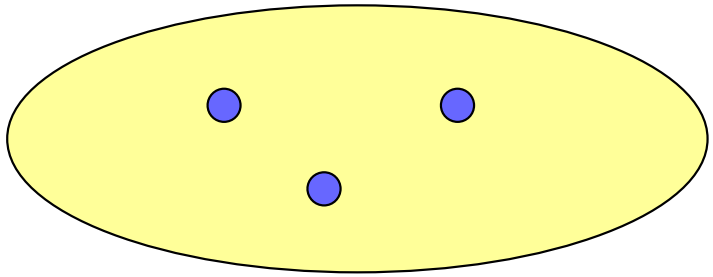
```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN  (SELECT  R.sid
                  FROM  Reserves R
                  WHERE  R.bid=103)
```

- In the previous example, the inner subquery is independent of the outer query.
- In general, the inner subquery could depend on the row currently being examined in the outer query.

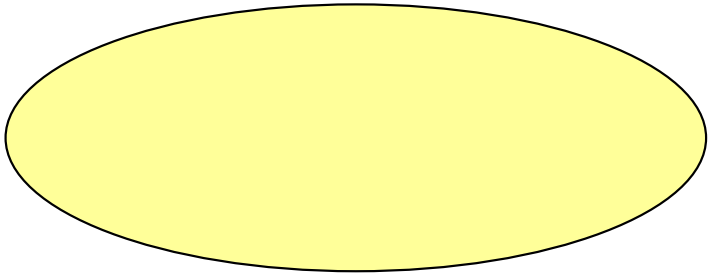
```
SELECT  S.sname
FROM  Sailors S
WHERE  1 <= (SELECT  COUNT(R.sid)
             FROM  Reserves R
             WHERE  R.bid=103 AND R.sid = S.sid)
```

Test for Empty Relations

- Word EXISTS returns TURE if the argument subquery is nonempty.

EXISTS()

TRUE

EXISTS()

FALSE

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q9': Find names of sailors who've reserved boat #103:

```
SELECT  S.sname
FROM  Sailors S
WHERE   EXISTS  (SELECT  *
                 FROM  Reserves R
                 WHERE  R.bid=103 AND S.sid=R.sid)
```

- **EXISTS** is another set comparison operator, which allows us to test whether a set is nonempty.
- * represents all attributes.

Set-Comparison Operators

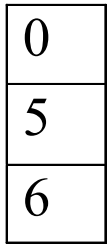
- We've already seen **IN** and **EXISTS**.

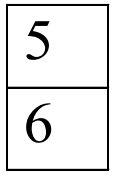
  We can also use **NOT IN** and **NOT EXISTS**.

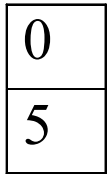
- Also available: ***op* ANY**, ***op* ALL**
  - Where op is one of the arithmetic comparison operator $\geq, \leq, <, =, >, \neq$
  - SOME is also available, but it is just a synonym for ANY.
- Q10: Find sailors whose rating is greater than that of some sailor called Tom:

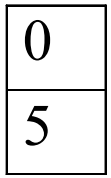
```
SELECT  *
FROM  Sailors S
WHERE  S.rating > ANY  (SELECT  S2.rating
                        FROM  Sailors S2
                        WHERE S2.sname='Tom')
```

SOME/ANY Semantics

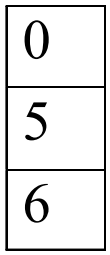
(5 > some ) returns TRUE (5 > 0)

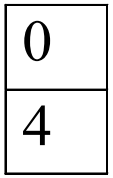
(5 > some ) returns FALSE

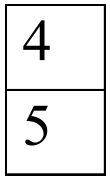
(5 = some ) = TRUE

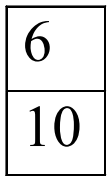
(5 ≠ some ) = TRUE (since 0 ≠ 5)

ALL Semantics

(5 > all ) = FALSE

(5 > all ) = TRUE

(5 = all ) = FALSE

(5 ≠ all ) = TRUE

Examples

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

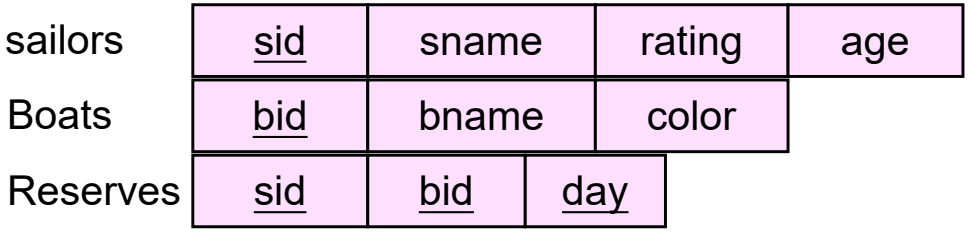
- Q11: Find sailors whose rating is better than every sailor called Tom.

```
SELECT  *
FROM  Sailors S
WHERE  S.rating > ALL  (SELECT  S2.rating
                        FROM  Sailors S2
                        WHERE S2.sname='Tom')
```

- Q12: Find the sailors with the highest rating.

```
SELECT  *
FROM  Sailors S
WHERE  S.rating >= ALL  (SELECT  S2.rating
                         FROM  Sailors S2)
```

Examples



- Rewriting INTERSECT queries using IN
- Q6': Find names of sailors who've reserved a red and a green boat

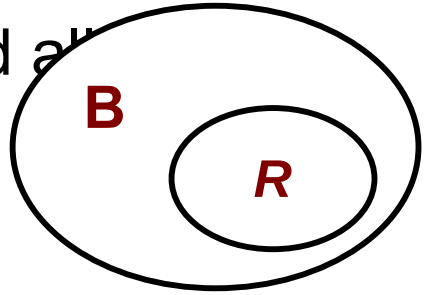
```
SELECT  S.name
FROM  Sailors S, Boats B, Reserves R
WHERE  S.sid=R.sid AND R.bid=B.bid AND B.color='red'
            AND S.sid IN  (SELECT  R2.sid
                                FROM Boats B2, Reserves R2
                                WHERE  R2.bid=B2.bid
                                            AND  B2.color='green')
```

- Similarly, EXCEPT queries can be re-written using NOT IN.

Division in SQL

sailors	sid	sname	rating	age
Boats	bid	bname	color	
Reserves	sid	bid	day	

- Q13: Find the names of sailors who've reserved a



NOT EXIST (B – R)

```
SELECT  S.sname
FROM  Sailors S
WHERE  NOT EXISTS
        ((SELECT  B.bid
          FROM  Boats B)
          EXCEPT
          (SELECT  R.bid
           FROM  Reserves R
           WHERE  R.sid=S.sid))
```

All boats

All boats reserved by S

- Note that this query is correlated – for each sailor S, we check to see that the set of boats reserved by S includes every boat.
- Logic: there does not exist any boat that is not reserved by S

A Working Example

Sailors

sid	sname	rating	age
s01	Tom	10.2	26
s02	James	20.3	38

Reserves

sid	bid	day
s01	b01	2003
s01	b02	2004
s02	b01	2002
s02	b02	2003
s02	b03	2004

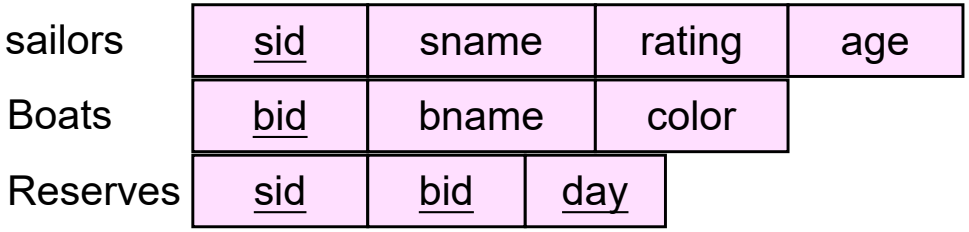
Boats

bid	bname	color
b01	Dragon	red
b02	Ocean	blue
b03	Roto	green

```
SELECT  S.sname
FROM  Sailors S
WHERE  NOT EXISTS
        ((SELECT  B.bid
          FROM  Boats B)
         EXCEPT
         (SELECT  R.bid
          FROM  Reserves R
          WHERE  R.sid=S.sid))
```

- All boats {b01, b02, b03}
- When sid=s01, then all the boats reserved by s01: {b01, b02}
- When sid=s02, then all the boats reserved by s01: {b01, b02, b03}

Division in SQL



- An Alternative way to write the previous query without using EXCEPT

```
SELECT  S.sname
FROM  Sailors S
WHERE  NOT EXISTS  (SELECT  B.bid
                    FROM  Boats B
                    WHERE  NOT EXISTS  (SELECT  R.bid
                                        FROM  Reserves R
                                        WHERE  R.bid=B.bid
                                               AND R.sid=S.sid))
```

- Intuitively, for each sailor we check that there is no boat that has not been reserved by this sailor.

Division in SQL

Sailors

sid	sname	rating	age
s01	Tom	10.2	26
s02	James	20.3	38

Boats

bid	bname	color
b01	Dragon	red
b02	Ocean	blue
b03	Roto	green

Reserves

sid	bid	day
s01	b01	2003
s01	b02	2004
s02	b01	2002
s02	b02	2003
s02	b03	2004

```
SELECT  S.sname
FROM  Sailors S
WHERE  NOT EXISTS  (SELECT  B.bid
                    FROM  Boats B
                    WHERE  NOT EXISTS  (SELECT  R.bid
                                        FROM  Reserves R
                                        WHERE  R.bid=B.bid
                                               AND R.sid=S.sid))
```